RAJALAKSHMI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

[Accredited by NAAC- "A++ Grade", Accredited by NBA, approved by AICTE &Govt. of Tamil Nadu, Affiliated to Anna University, Chennai]

Chennai-Bangalore Highway Road, Kuthambakkam, Chennai-600124

Department of Artificial Intelligence & Machine Learning

III Year/ V Semester

Regulation - 2023

AL23531 Natural Language Processing Laboratory Manual

Prepared by **Approved by**

N.Ajaypradeep
Assistant Professor
Department of Artificial Intelligence & Machine Learning

RAJALAKSHMI INSTITUTE OF TECHNOLOGY, CHENNAI
An Autonomous Institution, Affiliated to Anna University, Chennai

REGULATIONS 2023
CHOICE BASED CREDIT SYSTEM

B.E. COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

I VISION OF THE DEPARTMENT

- To encourage more employability, innovative research, and entrepreneurial thinking by developing emerging technology solutions for the benefit of industry and society through effective teaching and learning processes.

II MISSION OF THE DEPARTMENT

- Providing high-quality, value-added technical education and producing technology professionals who can think creatively and inspire others.
- Applying rational thought to design and create cutting-edge products in collaboration with industry stakeholders in order to meet global standards and demands.
- Improving essential skills in Artificial Intelligence and Machine Learning.

III PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- To excel in society, students will significantly advance their knowledge of the principles of engineering, science, and technology.
- To pursue higher education in an AI environment to address issues in the real world.
- To receive training in creating, developing, and quality assurance knowledge of AI and ML technologies in order to produce solutions to challenges encountered in the real world.
- To provide students with the knowledge and skills necessary to use computational tools and AI and ML effectively.
- To encourage graduates with advancing the knowledge of AI and ML approaches among entrepreneurs and researchers.

IV PROGRAM OUTCOMES (POs)

1. **Engineering Knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/Development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct Investigations of Complex Problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data and synthesis of the information to provide valid conclusions.
5. **Modern Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The Engineer and Society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and Sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and Team Work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project Management and Finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

V PROGRAM SPECIFIC OUTCOMES (PSOs)

The Students will be able to

1. Create and implement optimized solutions for computationally demanding applications.
2. Use Artificial Intelligence and Machine Learning Tools and Techniques to solve multidisciplinary problems
3. Create AI and Machine Learning models on data to facilitate better decision-making in societal issues.

AL23531	NATURAL LANGUAGE PROCESSING	L	T	P	C
		3	0	2	4

COURSE OBJECTIVES:

- To learn the fundamentals of natural language processing.
- To learn the word level analysis methods .
- To explore the syntactic analysis concepts.
- To understand the semantics and pragmatics.
- To Learn to analyze discourses and Lexical Resources

S.No	List of Experiment
1	Word Analysis
2	Word Generation
3	Morphology
4	N-Grams
5	N-Grams Smoothing
6	POS Tagging: Hidden Markov Model
7	POS Tagging: Viterbi Decoding
8	Building POS Tagger
9	Chunking
10	Building Chunker

Description

- Word Analysis Analyzing individual words in a text, including their frequency, length, distribution, etc.
- Word Generation: Generating new words, often based on certain linguistic rules or statistical models.
- Morphology: Studying the structure and formation of words, including prefixes, suffixes, and roots.
- N-Grams Sequences of N words occurring together in a text, used for various language modeling and prediction tasks.
- N-Grams Smoothing: Techniques used to address the issue of unseen N-grams in language models, improving their robustness.
- POS Tagging: Hidden Markov Model (HMM): Assigning part-of-speech tags to words in a sentence using a probabilistic model known as Hidden Markov Model.
- POS Tagging: Viterbi Decoding: An algorithm used in POS tagging to find the most likely sequence of POS tags for a given sequence of words, based on a probabilistic model.
- Building POS Tagger: Developing a program or model that automatically assigns part-of-speech tags to words in a given text.
- Chunking: Identifying and grouping together words or tokens into syntactically correlated chunks, such as noun phrases or verb phrases.
- Building Chunker: Creating a program or model that automatically identifies and labels chunks in a text.

COURSE OUTCOMES:

At the end of this course, the students will be able to:

CO1: Tag a given text with basic Language features

CO2 Implement a rule based system to tackle morphology/syntax of a language

CO3: Design a tag set to be used for statistical processing for real-time applications.

CO4: Compare and contrast the use of different statistical approaches for different types of NLP applications.

CO5: Use tools to process natural language and design innovative NLP applications.

TEXT BOOKS:

1. Daniel Jurafsky, James H. Martin—Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics and Speech, Pearson Publication, 2014.

2. Steven Bird, Ewan Klein and Edward Loper, —Natural Language Processing with Python, First Edition, O'Reilly Media, 2009.

REFERENCES:

1. Breck Baldwin, —Language Processing with Java and LingPipe Cookbook, Atlantic Publisher, 2015.

2. Richard M Reese, —Natural Language Processing with Java‖, O'Reilly Media, 2015.

.

EXP.No.1

Word Analysis Analyzing individual words in a text, including their frequency, length, distribution, etc.

AIM:

To analyze individual words in a text to understand their frequency, length, and distribution, thereby gaining insights into the text's vocabulary and linguistic patterns.

SOFTWARE AND HARDWARE SPECIFICATIONS:

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Convert to Lowercase**: Convert the input text to lowercase.
- **Tokenize**: Split the text into individual words.
- **Clean Words**: Remove punctuation from each word.
- **Count Frequencies**: Use a dictionary to count the frequency of each word.
- **Calculate Distribution**: Calculate the distribution of each word as its frequency divided by the total word count.
- **Sort and Print**: Sort words by frequency and print word, frequency, and distribution.

Program:

```
#word analysis
 def word_analysis(text):
    # Convert text to lowercase to ensure case-insensitive analysis
    text = text.lower()


 # Split the text into words
    words = text.split()

    # Initialize an empty dictionary to store word frequencies
    word_freq = {}

    # Iterate through each word in the text
    for word in words:
        # Remove any punctuation marks from the word
        word = word.strip('.,?!-')

        # If the word is already in the dictionary, increment its frequency count
        if word in word_freq:
            word_freq[word] += 1
        # If the word is not in the dictionary, add it with a frequency count of 1
        else:
            word_freq[word] = 1

    # Total number of words in the text
    total_words = sum(word_freq.values())

    # Calculate word distribution
    word_distribution = {word: freq/total_words for word, freq in word_freq.items()}

    # Sort the dictionary by frequency in descending order
    sorted_word_freq = sorted(word_freq.items(), key=lambda x: x[1], reverse=True)


# Print the word frequencies and distribution
    print("Word\t\tFrequency\tDistribution")
    print("-----------------------------------------")
    for word, freq in sorted_word_freq:
```

```
    print(f"{word.ljust(15)}{freq}\t\t{word_distribution[word]:.2%}")

# Example text for analysis
text = "This is a sample text for word analysis. Word analysis involves analyzing the frequency of each word in a given text."

# Perform word analysis on the example text
word_analysis(text)
```

OUTPUT:

```
Word                 Frequency       Distribution
----------------------------------------
word            3               14.29%
a               2               9.52%
text            2               9.52%
analysis        2               9.52%
this            1               4.76%
is              1               4.76%
sample          1               4.76%
for             1               4.76%
involves        1               4.76%
analyzing       1               4.76%
the             1               4.76%
frequency       1               4.76%
of              1               4.76%
each            1               4.76%
in              1               4.76%
given           1               4.76%
```

EXP.No.2

Word Generation: Generating new words, often based on certain linguistic rules or statistical models

AIM:

To generate new words based on certain linguistic rules or statistical models, which can be used for creative writing, language learning, or enhancing natural language processing (NLP) systems.

SOFTWARE AND HARDWARE SPECIFICATIONS:

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Define Rules/Model**: Define linguistic rules or a statistical model for word generation.
- **Set Parameters**: Set parameters like word length, starting letters, etc.
- **Generate Words**: Use the model to generate new words based on the defined rules.
- **Filter Results**: Filter the generated words for validity.
- **Store/Output**: Store or output the generated words.
- **Review and Refine**: Review the generated words and refine the model or rules as needed.

Program:

```
#word generation
import random

def generate_sentence(word_count, transition_probs):
    sentence = ""
    current_state = "@"

    for _ in range(word_count):
        word_length = random.randint(1, 10)  # Generate word lengths between 1 and 10
        word = generate_word(word_length, transition_probs)
        sentence += word + " "

    return sentence.strip()

def generate_word(word_length, transition_probs):
    word = ""
    current_state = "@"

    for _ in range(word_length):
        next_state = random.choices(
            list(transition_probs[current_state].keys()),
            weights=list(transition_probs[current_state].values())
        )[0]

        if next_state == "$":
            break

        word += next_state
        current_state = next_state

    return word

# Example transition probabilities for words
word_transition_probs = {
    "@": {"The": 0.3, "A": 0.5, "$": 0.2},
    "The": {"cat": 0.7, "dog": 0.3, "$": 0},
    "A": {"quick": 0.6, "lazy": 0.4, "$": 0},
```

```
    "quick": {"brown": 0.8, "black": 0.2, "$": 0},
    "lazy": {"brown": 1.0, "$": 0},
    "cat": {"jumped": 1.0, "$": 0},
    "dog": {"ran": 1.0, "$": 0},
    "brown": {"fox": 1.0, "$": 0},
    "black": {"dog": 1.0, "$": 0},
    "jumped": {"over": 1.0, "$": 0},
    "ran": {"through": 1.0, "$": 0},
    "over": {"the": 1.0, "$": 0},
    "through": {"the": 1.0, "$": 0},
    "the": {"fence": 1.0, "$": 0},
    "fence": {"$": 1.0}
}

# Generate a new sentence of word_count words based on the transition probabilities
new_sentence = generate_sentence(8, word_transition_probs)
print("Generated Sentence:", new_sentence)
```

OUTPUT:

Generated Sentence: Thedogranthrough Alazybrownfox Aquickbrown Thecat A Alazybrown

EXP.No.3

Morphology: Studying the structure and formation of words, including prefixes, suffixes, and roots

AIM:

To study the structure and formation of words by examining prefixes, suffixes, and roots, enabling a deeper understanding of word construction and etymology.

SOFTWARE AND HARDWARE SPECIFICATIONS:

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Tokenize Text**: Split the text into words.
- **Identify Morphemes**: Identify prefixes, suffixes, and roots in each word.
- **Classify Morphemes**: Classify each morpheme (prefix, root, suffix).
- **Analyse Structure**: Analyse the structure of each word based on its morphemes.

- **Store Data**: Store the morpheme data for each word.
- **Review Findings**: Review and summarize the morphological patterns found.

Program:

```
def extract_prefix_suffix(word):
    prefixes = []
    suffixes = []

    # Extract prefixes
    for i in range(1, len(word)):
        prefixes.append(word[:i])

    # Extract suffixes
    for i in range(len(word)-1, 0, -1):
        suffixes.append(word[i:])

    return prefixes, suffixes

# Example word
word = "prefixsuffix"

# Extract prefixes and suffixes
prefixes, suffixes = extract_prefix_suffix(word)

# Print prefixes and suffixes
print("Prefixes:", prefixes)
print("Suffixes:", suffixes)
```

Output:

Prefixes: ['p', 'pr', 'pre', 'pref', 'prefi', 'prefix', 'prefixs', 'prefixsu', 'prefixsuf', 'prefixsuff', 'prefixsuffi']
Suffixes: ['x', 'ix', 'fix', 'ffix', 'uffix', 'suffix', 'xsuffix', 'ixsuffix', 'fixsuffix', 'efixsuffix', 'refixsuffix']

EXP.No.4

AIM:

To create sequences of N words occurring together in a text, which can be used for language modeling, text prediction, and understanding contextual word usage.

SOFTWARE AND HARDWARE SPECIFICATIONS:

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Tokenize Text**: Split the text into words.
- **Create N-Grams**: Generate N-grams (sequences of N words) from the tokenized text.
- **Count Frequencies**: Count the frequency of each N-gram.
- **Calculate Probabilities**: Calculate the probability of each N-gram based on frequencies.
- **Store Results**: Store the N-gram data.
- **Analyse Patterns**: Analyse the patterns and use them for language modelling or prediction tasks.

Program:

```
import nltk
from nltk.util import ngrams
from collections import defaultdict, Counter
import math

# Sample text

text = "Natural language processing allows computers to understand human language."

# Tokenize the text
tokens = nltk.word_tokenize(text.lower())

# Generate bigrams
bigrams = list(ngrams(tokens, 2))

# Count frequencies of unigrams and bigrams
unigram_freq = Counter(tokens)
bigram_freq = Counter(bigrams)

# Calculate probabilities
bigram_prob = {bigram: bigram_freq[bigram] / unigram_freq[bigram[0]] for bigram in bigram_freq}

# Print bigram probabilities
print("Bigram Probabilities:")
for bigram, prob in bigram_prob.items():
    print(f"{bigram}: {prob:.4f}")

# Evaluate using Perplexity
def perplexity(sentence, bigram_prob, unigram_freq):
    tokens = nltk.word_tokenize(sentence.lower())
    bigrams = list(ngrams(tokens, 2))
    N = len(tokens)
    perplexity = 1
    for bigram in bigrams:
        if bigram in bigram_prob:
            perplexity *= 1 / bigram_prob[bigram]
        else:
```

```
        perplexity *= 1 / (unigram_freq[bigram[0]] + len(unigram_freq))  # Simple Laplace smoothing for unseen bigrams
    perplexity = pow(perplexity, 1/N)
    return perplexity

# Example sentence to evaluate
test_sentence = "Language processing allows understanding."



# Calculate perplexity
perplexity_score = perplexity(test_sentence, bigram_prob, unigram_freq)
print(f"Perplexity of the sentence '{test_sentence}': {perplexity_score:.4f}")

"""2. Part-of-Speech Tagging using NLTK
python

Copy code"""
import nltk
from nltk.corpus import treebank
from nltk.tag import hmm

# Load treebank data
training_data = treebank.tagged_sents()[:3000]
testing_data = treebank.tagged_sents()[3000:]

# Train HMM Tagger
trainer = hmm.HiddenMarkovModelTrainer()
hmm_tagger = trainer.train(training_data)

# Evaluate on test data
accuracy = hmm_tagger.evaluate(testing_data)
print(f"HMM Tagger Accuracy: {accuracy:.4f}")

# Tagging a sample sentence
sentence = "Natural language processing allows computers to understand human language.".split()
tagged_sentence = hmm_tagger.tag(sentence)
print("Tagged Sentence:")
print(tagged_sentence)
```

OUTPUT:

Bigram Probabilities:

('natural', 'language'): 1.0000

('language', 'processing'): 0.5000

('processing', 'allows'): 1.0000

('allows', 'computers'): 1.0000

('computers', 'to'): 1.0000

('to', 'understand'): 1.0000

('understand', 'human'): 1.0000

('human', 'language'): 1.0000

('language', '.'): 0.5000

Perplexity of the sentence 'Language processing allows understanding.': 0.4670

<ipython-input-24-35aff8c4e40d>:63: DeprecationWarning:

Function evaluate() has been deprecated. Use accuracy(gold)

instead.

accuracy = hmm_tagger.evaluate(testing_data)

/usr/local/lib/python3.10/dist-packages/nltk/tag/hmm.py:334: RuntimeWarning: overflow encountered in cast

X[i, j] = self._transitions[si].logprob(self._states[j])

/usr/local/lib/python3.10/dist-packages/nltk/tag/hmm.py:336: RuntimeWarning: overflow encountered in cast

O[i, k] = self._output_logprob(si, self._symbols[k])

/usr/local/lib/python3.10/dist-packages/nltk/tag/hmm.py:332: RuntimeWarning: overflow encountered in cast

P[i] = self._priors.logprob(si)

/usr/local/lib/python3.10/dist-packages/nltk/tag/hmm.py:364: RuntimeWarning: overflow encountered in cast

O[i, k] = self._output_logprob(si, self._symbols[k])

HMM Tagger Accuracy: 0.3684

Tagged Sentence:

[('Natural', 'NNP'), ('language', 'NN'), ('processing', 'NNP'), ('allows', 'NNP'), ('computers', 'NNP'), ('to', 'NNP'), ('understand', 'NNP'), ('human', 'NNP'), ('language.', 'NNP')]

EXP.No.5

Techniques used to address the issue of unseen N-grams in language models, improving their robustness.

AIM:

To apply techniques to address the issue of unseen N-grams in language models, improving the robustness and accuracy of these models in handling rare or previously unseen word combinations.

SOFTWARE AND HARDWARE SPECIFICATIONS:

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Generate N-Grams**: Generate N-grams from the text.
- **Count Frequencies**: Count the frequency of each N-gram.
- **Apply Smoothing**: Apply a smoothing technique (e.g., Laplace smoothing) to handle unseen N-grams.
- **Recalculate Probabilities**: Recalculate the probabilities of N-grams after smoothing.
- **Store Smoothed Data**: Store the smoothed N-gram data.
- **Validate Model**: Validate the robustness of the language model with smoothed N-grams.

Program:

```
import nltk
from collections import Counter

# Sample text
text = "Natural language processing allows computers to understand human language."

# Tokenize the text
tokens = nltk.word_tokenize(text.lower())

# Generate bigrams
bigrams = list(nltk.bigrams(tokens))

# Count frequencies of unigrams and bigrams
unigram_freq = Counter(tokens)
bigram_freq = Counter(bigrams)

# Vocabulary size
V = len(unigram_freq)

# Laplace Smoothed Probability
def laplace_smoothed_prob(bigram, bigram_freq, unigram_freq, V):
    return (bigram_freq[bigram] + 1) / (unigram_freq[bigram[0]] + V)

# Example calculation for a bigram
example_bigram = ('language', 'processing')
prob = laplace_smoothed_prob(example_bigram, bigram_freq, unigram_freq, V)
print(f"Laplace Smoothed Probability of {example_bigram}: {prob:.4f}")
def additive_smoothed_prob(bigram, bigram_freq, unigram_freq, V, alpha):
    return (bigram_freq[bigram] + alpha) / (unigram_freq[bigram[0]] + alpha * V)

# Example with alpha = 0.5
alpha = 0.5
prob = additive_smoothed_prob(example_bigram, bigram_freq, unigram_freq, V, alpha)
print(f"Additive Smoothed Probability of {example_bigram} with alpha={alpha}: {prob:.4f}")
def good_turing_prob(bigram, bigram_freq, unigram_freq):
    # Frequency of frequencies
    freq_of_freqs = Counter(bigram_freq.values())
```

```
    # Adjusted count
    c = bigram_freq[bigram]
    c_star = (c + 1) * (freq_of_freqs[c + 1] / freq_of_freqs[c]) if freq_of_freqs[c] != 0 else 0

    # Total number of bigrams
    N = sum(bigram_freq.values())

    return c_star / N

# Example calculation
prob = good_turing_prob(example_bigram, bigram_freq, unigram_freq)
print(f"Good-Turing Probability of {example_bigram}: {prob:.4f}")
from nltk.lm import MLE
from nltk.lm.models import KneserNeyInterpolated
from nltk.util import pad_sequence, bigrams, everygrams
from nltk.lm.preprocessing import padded_everygram_pipeline

# Sample text as training data
train_data = [['natural', 'language', 'processing', 'allows', 'computers', 'to', 'understand', 'human',
'language']]
n = 2  # For bigrams

# Prepare the data
train_data, padded_sents = padded_everygram_pipeline(n, train_data)

# Train Kneser-Ney model
kn_model = KneserNeyInterpolated(n)
kn_model.fit(train_data, padded_sents)

# Example probability
prob = kn_model.score('processing', ['language'])
print(f"Kneser-Ney Smoothed Probability: {prob:.4f}")
from nltk.lm import MLE
from nltk.lm.models import WittenBellInterpolated
from nltk.util import pad_sequence, everygrams
from nltk.lm.preprocessing import padded_everygram_pipeline

# Sample text as training data
train_data = [['natural', 'language', 'processing', 'allows', 'computers', 'to', 'understand', 'human',
'language']]
```

```
n = 2  # For bigrams

# Prepare the data
train_data, padded_sents = padded_everygram_pipeline(n, train_data)

# Train Witten-Bell model (an example of interpolation)
wb_model = WittenBellInterpolated(n)
wb_model.fit(train_data, padded_sents)

# Example probability
prob = wb_model.score('processing', ['language'])
print(f"Witten-Bell Interpolated Probability: {prob:.4f}")
```

Output:

```
Laplace Smoothed Probability of ('language', 'processing'): 0.1818
Additive Smoothed Probability of ('language', 'processing') with alpha=0.5: 0.2308
Good-Turing Probability of ('language', 'processing'): 0.0000
Kneser-Ney Smoothed Probability: 0.4600
Witten-Bell Interpolated Probability: 0.2955
```

EXP.No.6

Assigning part-of-speech tags to words in a sentence using a probabilistic model known as Hidden Markov Model.

AIM:

To assign part-of-speech tags to words in a sentence using a probabilistic model known as the Hidden Markov Model, enhancing the understanding of the grammatical structure of the text.

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Tokenize Text**: Split the text into sentences and words.
- **Prepare Training Data**: Use a labeled corpus to prepare training data.
- **Train HMM**: Train an HMM on the training data to learn POS tags.
- **Apply HMM**: Use the trained HMM to predict POS tags for new text.
- **Evaluate Model**: Evaluate the accuracy of the HMM.
- **Refine Model**: Refine the model based on evaluation results.

Program:

```
import nltk
from nltk.corpus import treebank

# Download the necessary datasets
nltk.download('treebank')
nltk.download('universal_tagset')

# Load Treebank corpus
train_data = treebank.tagged_sents(tagset='universal')[:3000]
test_data = treebank.tagged_sents(tagset='universal')[3000:]
from nltk.tag.hmm import HiddenMarkovModelTrainer

# Train HMM Tagger
trainer = HiddenMarkovModelTrainer()
hmm_tagger = trainer.train(train_data)

# Evaluate on test data
accuracy = hmm_tagger.evaluate(test_data)
print(f"HMM Tagger Accuracy: {accuracy:.4f}")
# Tagging a sample sentence
sentence = "Natural language processing allows computers to understand human language.".split()
tagged_sentence = hmm_tagger.tag(sentence)
print("Tagged Sentence:")
print(tagged_sentence)
import nltk
from nltk.corpus import treebank
from nltk.tag.hmm import HiddenMarkovModelTrainer

# Download the necessary datasets
nltk.download('treebank')
nltk.download('universal_tagset')

# Load Treebank corpus
train_data = treebank.tagged_sents(tagset='universal')[:3000]
test_data = treebank.tagged_sents(tagset='universal')[3000:]

# Train HMM Tagger
```

```
trainer = HiddenMarkovModelTrainer()
hmm_tagger = trainer.train(train_data)

# Evaluate on test data
accuracy = hmm_tagger.evaluate(test_data)
print(f"HMM Tagger Accuracy: {accuracy:.4f}")

# Tagging a sample sentence
sentence = "Natural language processing allows computers to understand human language.".split()
tagged_sentence = hmm_tagger.tag(sentence)


print("Tagged Sentence:")
print(tagged_sentence)
```

OUTPUT:

```
[nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data]   Package treebank is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!
<ipython-input-26-0e0085359ea3>:
 HMM Tagger Accuracy: 0.5160
Tagged Sentence:
[('Natural', 'NOUN'), ('language', 'NOUN'), ('processing', 'NOUN'), ('allows', 'NOUN'), ('computers',
'NOUN'), ('to', 'NOUN'), ('understand', 'NOUN'), ('human', 'NOUN'), ('language.', 'NOUN')]
```

EXP.No.7

An algorithm used in POS tagging to find the most likely sequence of POS tags for a given sequence of words, based on a probabilistic model.

AIM:

To assign part-of-speech tags to words in a sentence using a probabilistic model known as the Hidden Markov Model, enhancing the understanding of the grammatical structure of the text.

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Tokenize Text**: Split the text into sentences and words.
- **Prepare HMM**: Ensure you have a trained HMM with emission and transition probabilities.
- **Initialize Viterbi Matrix**: Initialize the Viterbi matrix and backpointer matrix.
- **Fill Viterbi Matrix**: Use the Viterbi algorithm to fill the matrix based on the probabilities.
- **Backtrace**: Backtrace to find the most likely sequence of POS tags.
- **Output Tags**: Output the POS tags for the given text

Program:

```
import nltk
from nltk.tag import hmm
from nltk.corpus import treebank

# Ensure the necessary data is downloaded
```

```
nltk.download('treebank')

nltk.download('universal_tagset')

# Load the Penn Treebank corpus with the universal tagset
train_data = treebank.tagged_sents(tagset='universal')

# Train an HMM POS tagger
trainer = hmm.HiddenMarkovModelTrainer()
hmm_tagger = trainer.train(train_data)

def pos_tag_sentence(sentence):
    # Tokenize the sentence
    tokens = nltk.word_tokenize(sentence)
    # Perform POS tagging using the Viterbi algorithm
    pos_tags = hmm_tagger.tag(tokens)
    return pos_tags

# Example usage
sentence = "The quick brown fox jumps over the lazy dog."
pos_tags = pos_tag_sentence(sentence)
print(pos_tags)
```

OUTPUT:

```
[nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data]   Unzipping corpora/treebank.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Unzipping taggers/universal_tagset.zip.
/usr/local/lib/python3.10/dist-packages/nltk/tag/hmm.py:336: RuntimeWarning: overflow
encountered in cast
  O[i, k] = self._output_logprob(si, self._symbols[k])
[('The', 'DET'), ('quick', 'ADJ'), ('brown', 'NOUN'), ('fox', 'NOUN'), ('jumps', 'NOUN'), ('over',
'NOUN'), ('the', 'NOUN'), ('lazy', 'NOUN'), ('dog', 'NOUN'), ('.', 'NOUN')]
```

EXP.No.8

Developing a program or model that automatically assigns part-of-speech tags to words in a given text.

AIM:

To develop a program or model that automatically assigns part-of-speech tags to words in a given text, facilitating various NLP tasks such as parsing, information extraction, and text analysis.

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Collect Data**: Collect and preprocess a labeled corpus.
- **Extract Features**: Extract features relevant for POS tagging.
- **Choose Model**: Select a model (e.g., HMM, CRF).
- **Train Model**: Train the model on the labeled data.
- **Evaluate Model**: Evaluate the model's performance on a test set.
- **Deploy Model**: Deploy the model for tagging new text.

Program:

```
import nltk
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt')
text=nltk.word_tokenize("and now for everything completely same")
nltk.pos_tag(text)
```

OUTPUT:

```
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[('and', 'CC'),
 ('now', 'RB'),
 ('for', 'IN'),
 ('everything', 'NN'),
 ('completely', 'RB'),
 ('same', 'JJ')]
```

EXP.No.9

Identifying and grouping together words or tokens into syntactically correlated chunks, such as noun phrases or verb phrases.

AIM:

To identify and group together words or tokens into syntactically correlated chunks, such as noun phrases or verb phrases, aiding in higher-level syntactic and semantic analysis.

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Tokenize Text**: Split the text into sentences and words.
- **POS Tagging**: Assign POS tags to the words.
- **Define Chunk Rules**: Define rules or patterns for identifying chunks (e.g., noun phrases).
- **Apply Rules**: Apply the rules to tag chunks in the text.
- **Evaluate Chunks**: Evaluate the accuracy of the chunking.
- **Refine Rules**: Refine the chunking rules based on evaluation results.

PROGRAM:

```
import nltk
sentence=[("the","DT"),("little","JJ"),("yellow","JJ"),("dog","NN"),("barked","VBD"),("at","IN"),("the","DT"),("cat","NN")]
grammer="NP:{<DT>?<JJ>*<NN>}"
cp=nltk.RegexpParser(grammer)
result=cp.parse(sentence)
print(result)
```

OUTPUT:

```
(S
  (NP the/DT little/JJ yellow/JJ dog/NN)
  barked/VBD
  at/IN
  (NP the/DT cat/NN))
```

EXP.No.10

Creating a program or model that automatically identifies and labels chunks in a text.

AIM:

To create a program or model that automatically identifies and labels chunks in a text, improving the understanding of sentence structure and enhancing tasks like information extraction and parsing.

Software Specifications:

- Anaconda Navigator (Python, NLTK (Natural Language Toolkit), or any other programming language and libraries for text Processing)
- Google Colab

Hardware Specifications:

- Windows 10/11
- RAM - 16 GB
- Hard-disk - 1 TB
- Processor - Intel i5/i7

Algorithm:

- **Collect Data**: Collect and preprocess a labeled chunking corpus.
- **Extract Features**: Extract features relevant for chunking.
- **Choose Model**: Select a model (e.g., CRF, decision tree).
- **Train Model**: Train the model on the labeled data.
- **Evaluate Model**: Evaluate the model's performance on a test set.
- **Deploy Model**: Deploy the model for chunking new text.

Program:

```
import nltk
from nltk import pos_tag
from nltk.tokenize import word_tokenize
from nltk.chunk import RegexpParser

# Download necessary datasets

nltk.download('averaged_perceptron_tagger')
nltk.download('punkt')

# Sample sentence
sentence = "The quick brown fox jumps over the lazy dog"

# Tokenize the sentence
tokens = word_tokenize(sentence)

# Tag the tokens with POS tags
tagged_tokens = pos_tag(tokens)

print("POS Tagged Tokens:", tagged_tokens)

# Define chunk grammar for noun phrases
chunk_grammar = r"""
    NP: {<DT>?<JJ>*<NN>}    # Chunk optional determiner, adjectives, and a noun
"""

# Create a chunk parser
chunk_parser = RegexpParser(chunk_grammar)

# Parse the tagged tokens to find noun phrases
chunked_sentence = chunk_parser.parse(tagged_tokens)

print("Chunked Sentence:", chunked_sentence)

# Visualize the chunked sentence
#chunked_sentence.draw()
```

OUTPUT:

POS Tagged Tokens: [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN')]

Chunked Sentence: (S

(NP The/DT quick/JJ brown/NN)

(NP fox/NN)

jumps/VBZ

over/IN

(NP the/DT lazy/JJ dog/NN))

[nltk_data] Downloading package averaged_perceptron_tagger to

[nltk_data] /root/nltk_data...

[nltk_data] Package averaged_perceptron_tagger is already up-to-

[nltk_data] date!

[nltk_data] Downloading package punkt to /root/nltk_data...

[nltk_data] Package punkt is already up-to-date!